Carleton University
Department of Systems and Computer Engineering

SYSC 3303A

Real-Time Concurrent Systems — Winter 2026

Firefighting Drone Swarm

Final Project Report

Group 5

Name	Student ID
Faraaz	101298165
Sabateesh	101259947
Aymen	101326354
Ibrahim	101235067

April 9, 2026

Contents

1 Introduction **2**

2 Breakdown of Responsibilities **2**
2.1 Iteration 0 — Data Collection . . . 2
2.2 Iteration 1 — Basic Communication . . . 2
2.3 Iteration 2 — Scheduling & Drone State Machines . . . 3
2.4 Iteration 3 — Multiple Drones & Distributed Execution . . . 3
2.5 Iteration 4 — Fault Handling . . . 3
2.6 Iteration 5 — Capacity Limits, GUI & Metrics . . . 3

3 Diagrams **4**
3.1 UML Class Diagram . . . 4
3.2 State Machine Diagram — Scheduler . . . 5
3.3 State Machine Diagram — Drone . . . 5
3.4 State Machine Diagram — Drone (Updated with Faults) . . . 6
3.5 Sequence Diagram — Deploy Drone . . . 7
3.6 Sequence Diagram — Initialization . . . 8
3.7 Sequence Diagram — Refill . . . 9
3.8 Sequence Diagrams — Error Scenarios . . . 10
3.9 Error Scenario 1: Drone Stuck Mid-Flight . . . 10
3.10 Error Scenario 2: Nozzle Jammed . . . 11
3.11 Error Scenario 3: Packet Loss . . . 12
3.12 Timing Diagrams for the Scheduler . . . 13

4 Detailed Setup and Test Instructions **14**
4.1 Prerequisites . . . 14
4.2 Running the Simulation . . . 14
4.3 Testing Fault Scenarios . . . 14
4.3.1 CSV-Based Fault Injection . . . 14
4.3.2 GUI-Based Fault Injection . . . 14
4.4 Unit Tests . . . 15
4.5 Test Input Files . . . 15

5 Results from Measurements **15**
5.1 Required Performance Metrics . . . 15
5.2 Simulation Overview . . . 16

6 Design Reflection **17**
6.1 What We Like . . . 17
6.2 What Should Be Redone/Improved . . . 18

1 Introduction

This report documents the design, implementation, and testing of a firefighting drone swarm system projected that was developed over five iterations for SYSC 3303A. The system consists of three distributed subsystems communicating over UDP:

1. **Scheduler** — Central coordinator that receives fire incident requests, dispatches drones, monitors status, detects faults, and manages mission queues.
2. **Drone Subsystem** — Manages multiple drone threads, each implementing a state machine for flight, agent delivery, refilling, and fault handling.
3. **Fire Incident Subsystem** — Reads fire events from a CSV input file and sends them to the Scheduler with configurable timing and fault injection.

Each subsystem runs as a separate Java process, communicating exclusively via `DatagramSocket` (UDP). The system operates in simulated real-time with configurable parameters including the number of drones, number of zones, travel speed, agent capacity, and simulation speed. The final iteration supports 20 drones across 5+ fire zones with full fault handling, a real-time GUI, and automatic performance metrics collection.

2 Breakdown of Responsibilities

2.1 Iteration 0 — Data Collection

Member	Responsibilities
Faraaz (101298165)	Data Collection for Drone 2
Sabateesh (101259947)	Data Collection for Drone 1
Ibrahim (101235067)	Data Collection for Drone 3
Aymen (101326354)	Data Collection for Drone 4

2.2 Iteration 1 — Basic Communication

Member	Responsibilities
Faraaz (101298165)	Documentation, diagrams, fixing bugs and code restructuring
Sabateesh (101259947)	FireIncidentSubsystem, FireIncidentSubsystem tests
Ibrahim (101235067)	Scheduler, SchedulerServer
Aymen (101326354)	DroneSubsystem, Drone

2.3 Iteration 2 — Scheduling & Drone State Machines

Member	Responsibilities
Faraaz (101298165)	Scheduling logic, work on GUI
Sabateesh (101259947)	Main GUI development, tests for DroneStateMachine
Ibrahim (101235067)	Making diagrams, state machine planning
Aymen (101326354)	Drone state transitions, GUI interaction setup

2.4 Iteration 3 — Multiple Drones & Distributed Execution

Member	Responsibilities
Faraaz (101298165)	UDP communication between Scheduler and subsystems, UDP message test class, updates to GUI based on Scheduler datagrams
Sabateesh (101259947)	UML sequence and class diagrams, GUI updates to information based on fire info, display drone path
Aymen (101326354)	Update Drone Subsystem UDP communications, packet handling and registration with Scheduler
Ibrahim (101235067)	GUI visual changes and reorganization, updated GUI to show drone position, animations for visual feedback

2.5 Iteration 4 — Fault Handling

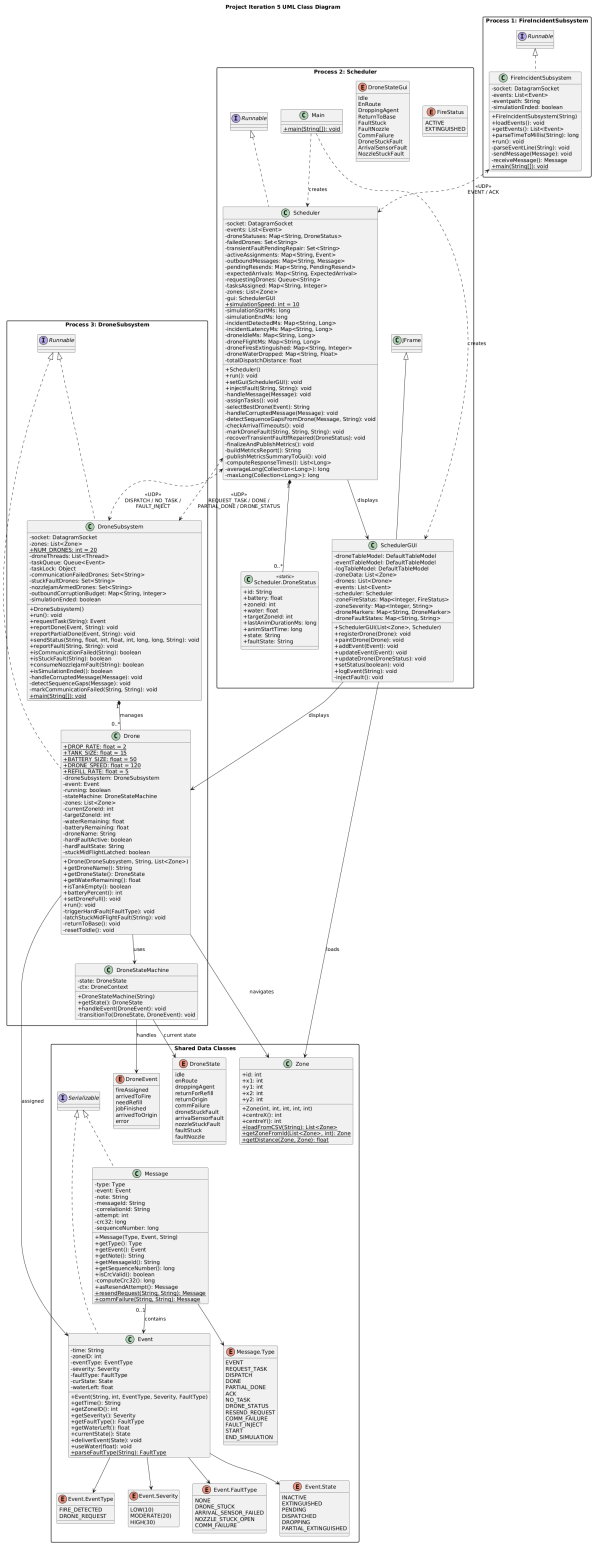
Member	Responsibilities
Faraaz (101298165)	Implemented drone faults and drone fault detection, CRC checksum redundancy in UDP messages, marking drones as faulty when detected
Sabateesh (101259947)	GUI changes, updated GUI to show drone fault handling
Aymen (101326354)	Diagrams, updated drone state machine, update Drone Subsystem UDP communications, packet handling and registration with Scheduler
Ibrahim (101235067)	Drone fault handling, returning faulty drones to base station, redistributing tasks when drone failure occurs

2.6 Iteration 5 — Capacity Limits, GUI & Metrics

Member	Responsibilities
Faraaz (101298165)	Added simulation metrics recording, fixed fault detection for nozzle jam and stuck mid-flight, fault info displayed on GUI, drone real-time location
Sabateesh (101259947)	Fault fixes, final report made new diagrams, Added more metrics to data collection log
Aymen (101326354)	Timing diagrams
Ibrahim (101235067)	Timing diagrams

3 Diagrams

3.1 UML Class Diagram



3.2 State Machine Diagram — Scheduler

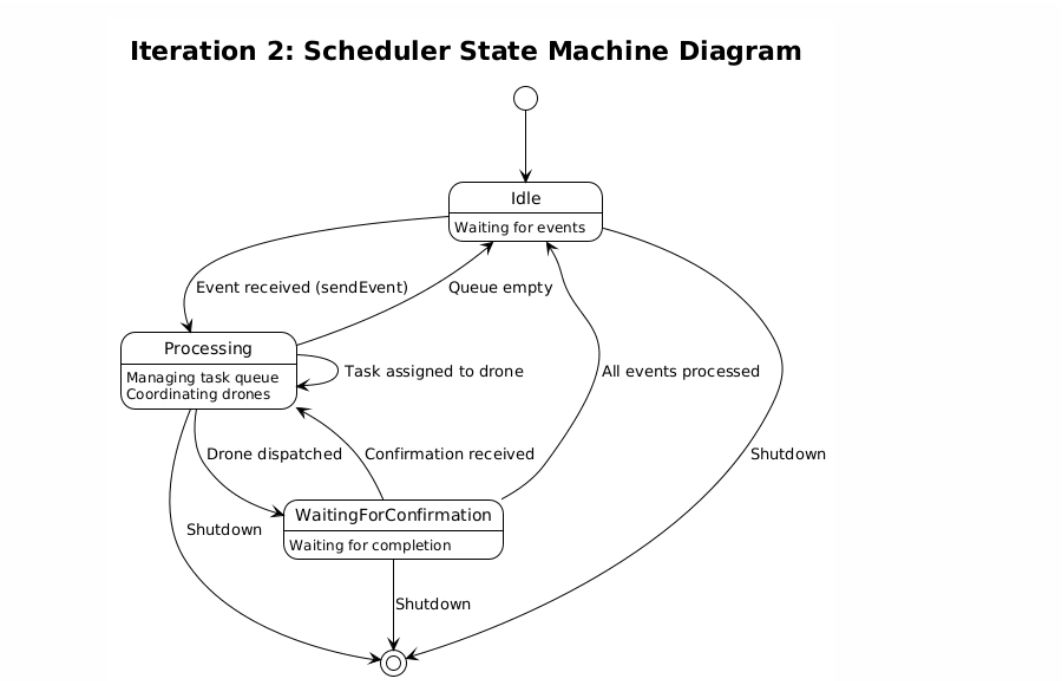


Figure 1: Scheduler State Machine Diagram.

3.3 State Machine Diagram — Drone

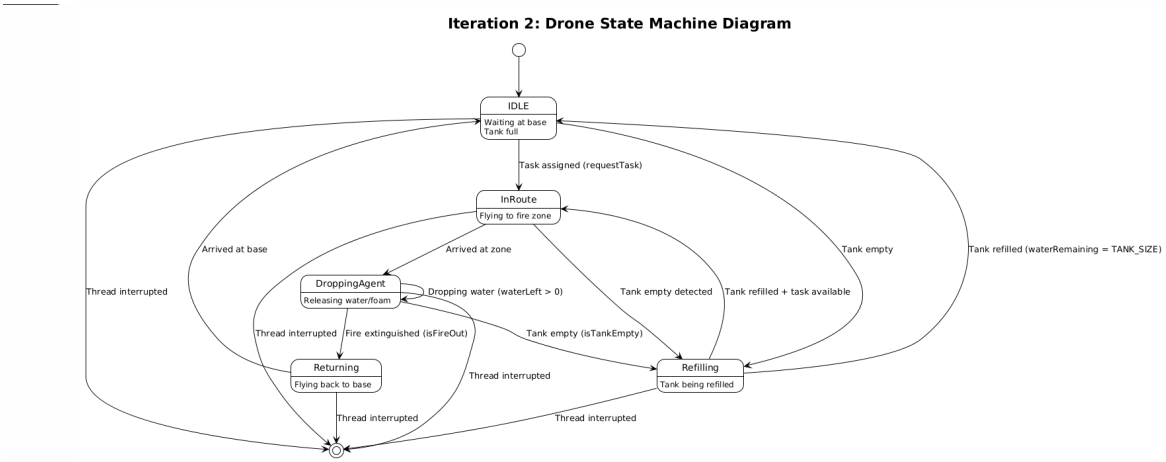


Figure 2: Drone State Machine Diagram (Iteration 2).

3.4 State Machine Diagram — Drone (Updated with Faults)

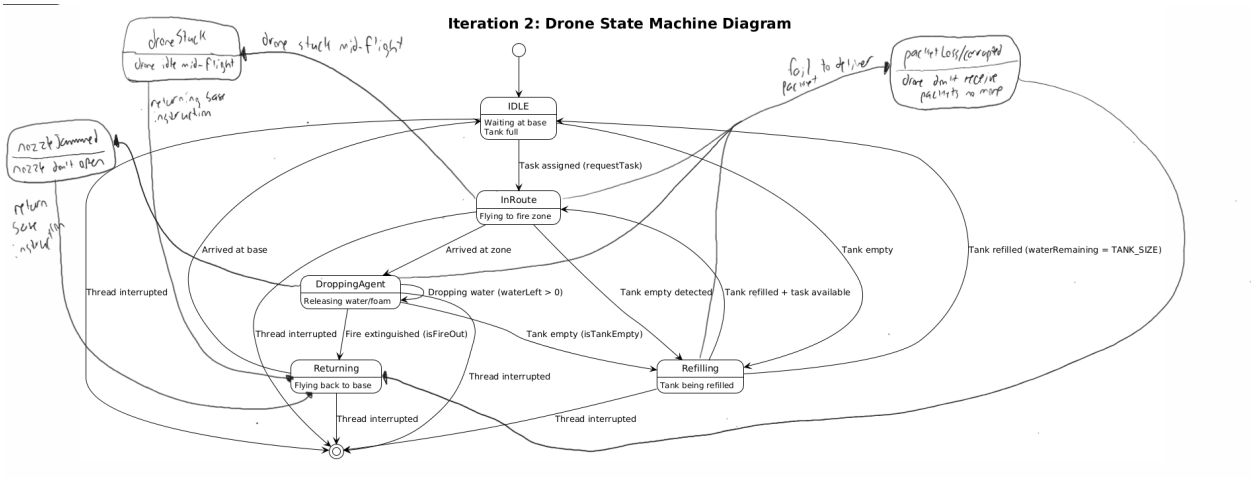


Figure 3: Drone State Machine Diagram updated for Iteration 4 with fault states: drone stuck mid-flight, nozzle jammed, and packet loss/corrupted message.

3.5 Sequence Diagram — Deploy Drone

Figure 4: Sequence diagram for the Deploy Drone use case.

3.6 Sequence Diagram — Initialization

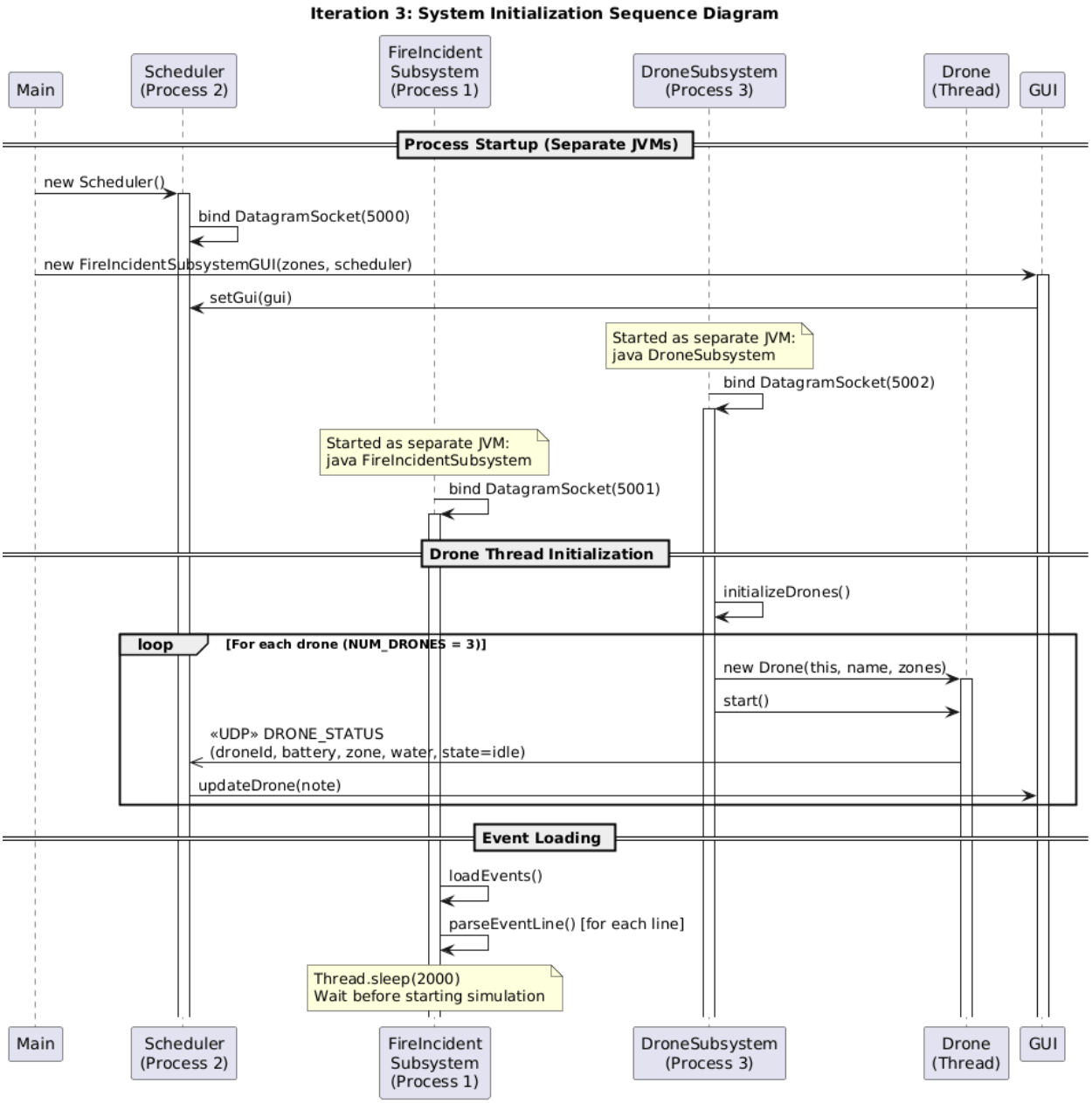


Figure 5: Sequence diagram for system initialization.

3.7 Sequence Diagram — Refill

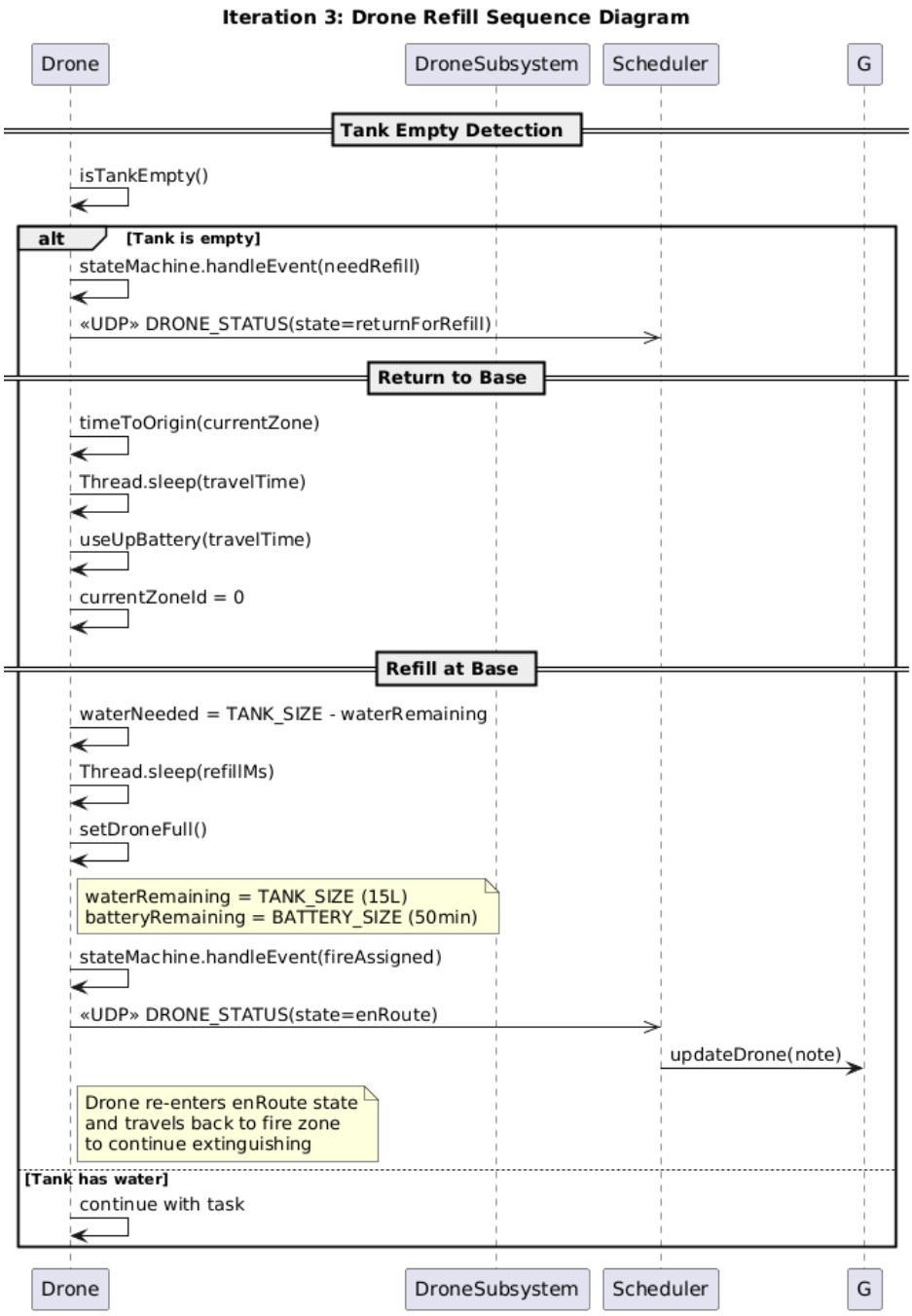


Figure 6: Sequence diagram for the drone refill scenario.

3.8 Sequence Diagrams — Error Scenarios

3.9 Error Scenario 1: Drone Stuck Mid-Flight

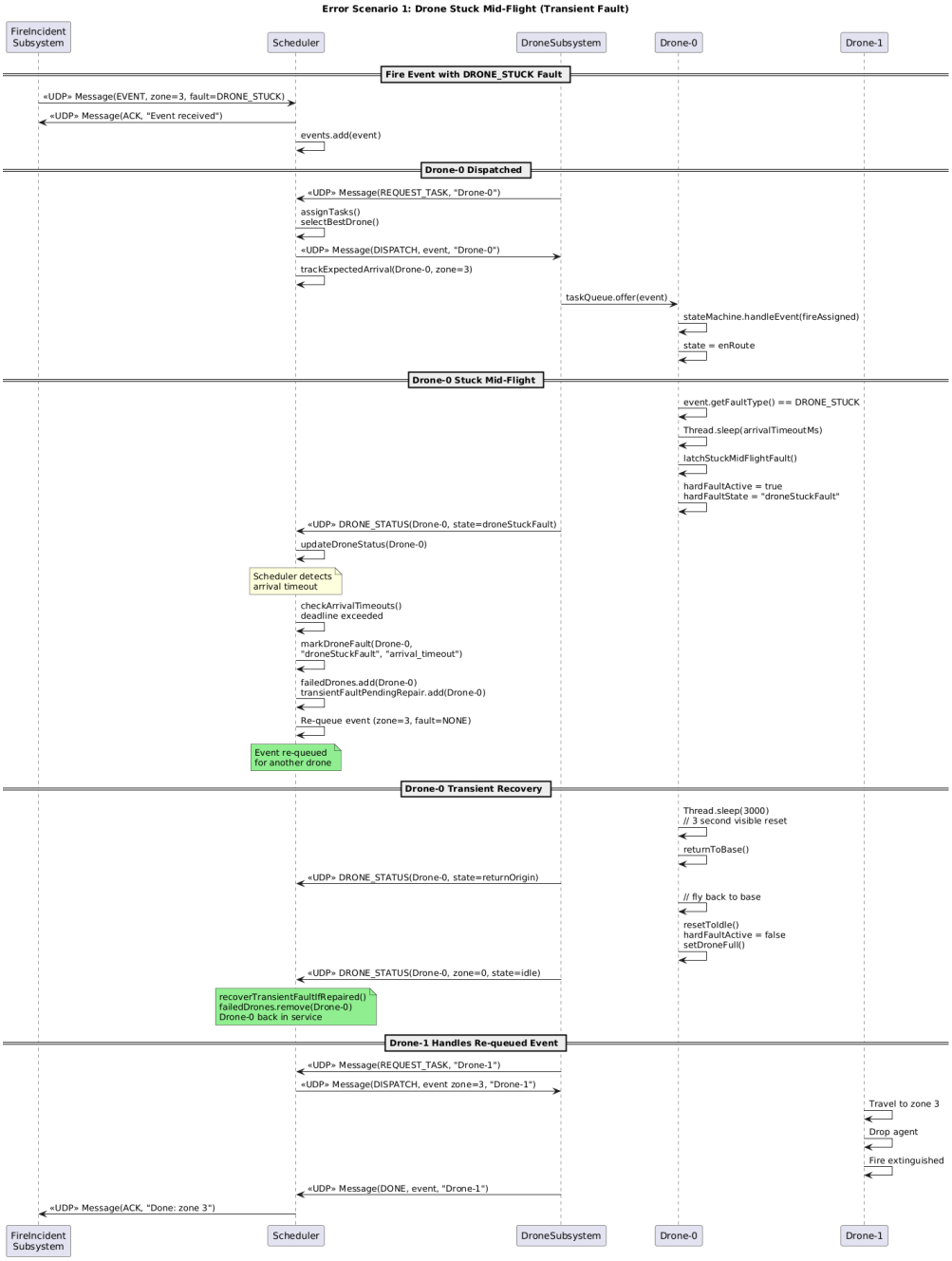


Figure 7: Sequence Diagram for drone getting stuck mid flight

3.10 Error Scenario 2: Nozzle Jammed

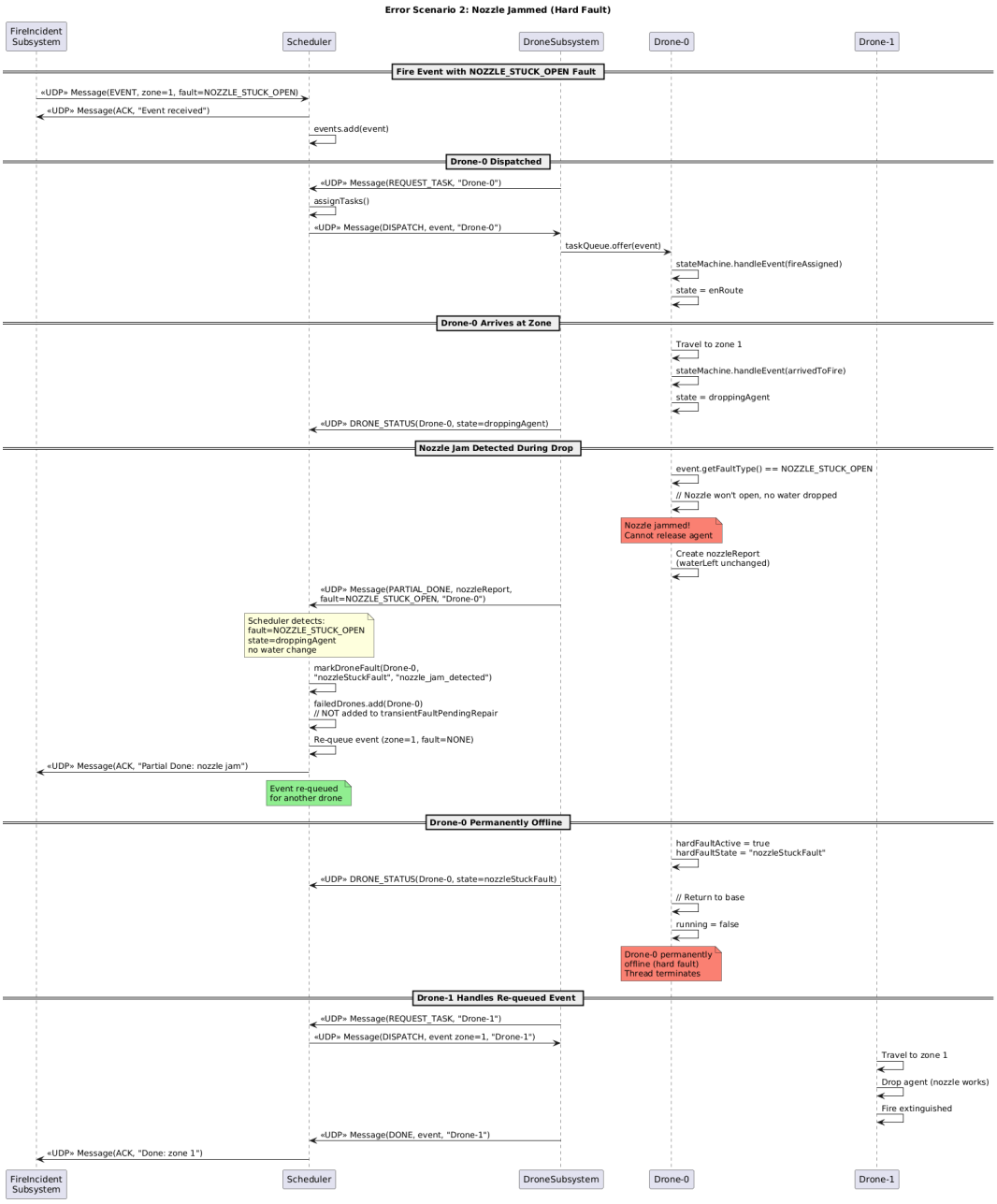


Figure 8: Sequence Diagram for nozzle getting jammed

3.11 Error Scenario 3: Packet Loss

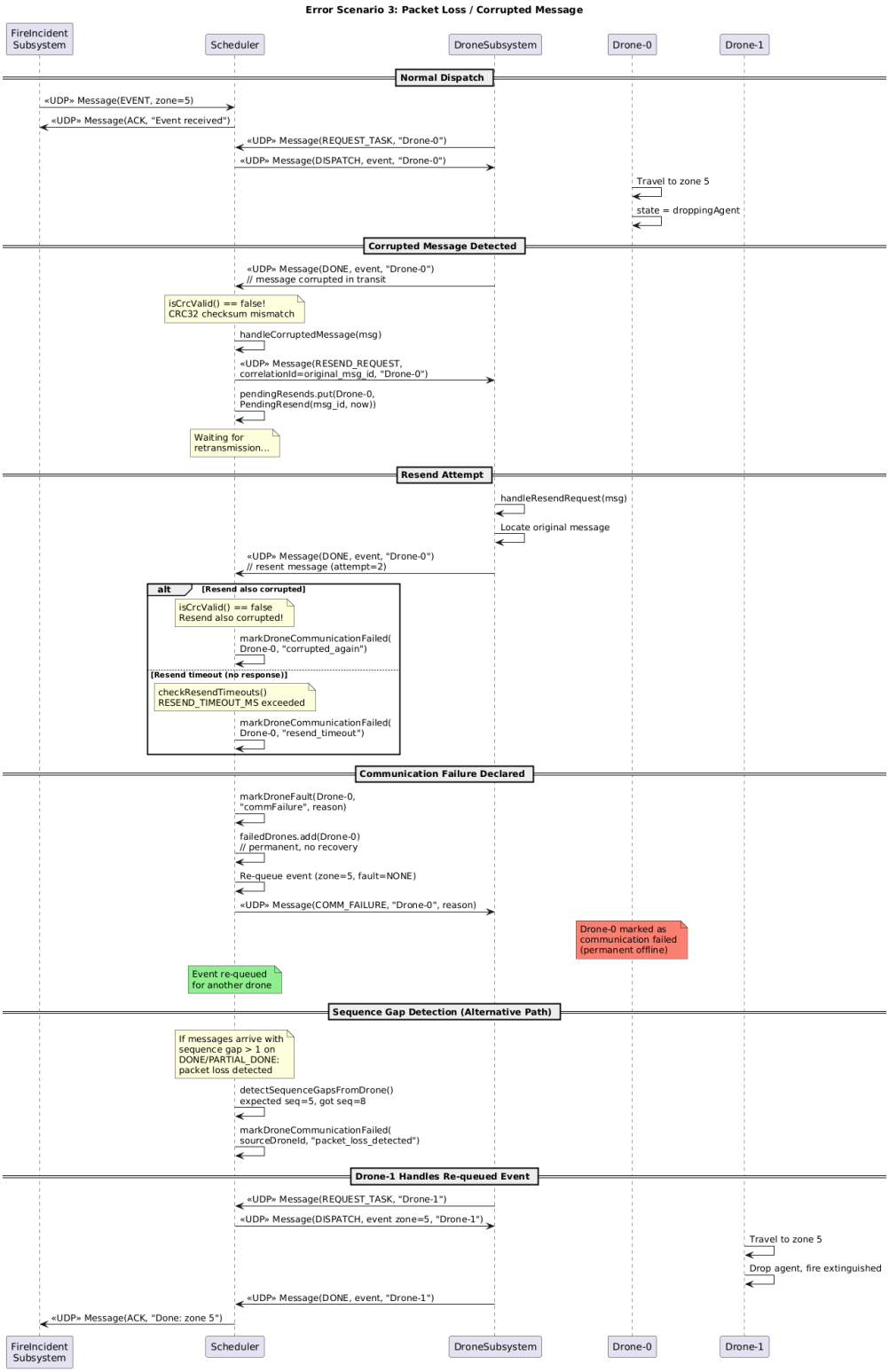


Figure 9: Sequence Diagram for Packet Loss

3.12 Timing Diagrams for the Scheduler

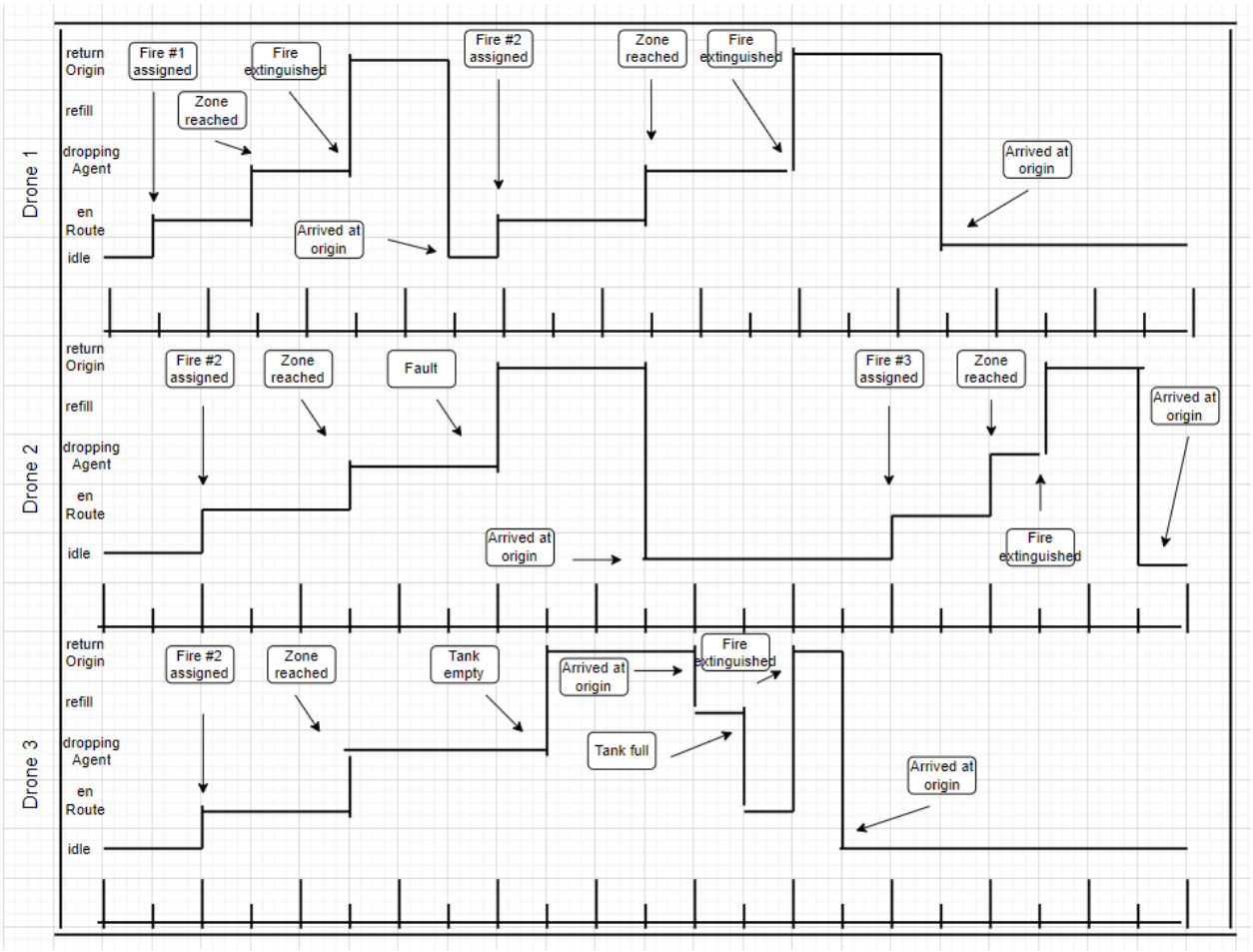


Figure 10: Timing diagram showing three drones operating concurrently. Drone 1 handles two fires. Drone 2 encounters a fault mid-flight and is later reassigned. Drone 3 runs out of agent, refills at base, then returns to finish the fire.

The additional timing diagrams for the three fault scenarios are provided in the accompanying `PI4_TimingDiagrams.pdf`. Each diagram shows the timeline of events across all subsystems with color-coded state bars and message arrows.

- **Diagram 1:** Drone Stuck Mid-Flight — transient fault, timeout detection, Scheduler reroute
- **Diagram 2:** Nozzle Jammed — hard fault, drone offline, Scheduler reroute
- **Diagram 3:** Packet Loss / Corrupted Message — CRC failure, resend, communication failure

4 Detailed Setup and Test Instructions

4.1 Prerequisites

- Java 17 or higher
- IntelliJ IDEA (recommended)

4.2 Running the Simulation

1. Open the project in IntelliJ IDEA.
2. Ensure the test files `event_file.csv` and `zone_file.csv`
3. **Run** `Main.java` — Starts the Scheduler and opens the GUI.
4. **Run** `DroneSubsystem.java` — Starts the Drone Subsystem with all drone threads.
5. **Run** `FireIncidentSubsystem.java` — Reads the CSV and begins sending events.

Important: Start in order: Main → DroneSubsystem → FireIncidentSubsystem.

4.3 Testing Fault Scenarios

4.3.1 CSV-Based Fault Injection

Edit `event_file.csv` to include faults in the 5th column. Valid values: `NONE`, `DRONE_STUCK`, `ARRIVAL_SENSOR_FAILED`, `NOZZLE_STUCK_OPEN`.

Expected behavior:

- **DRONE_STUCK**: Drone gets stuck mid-flight, resets after 3 seconds, returns to base, re-enters service. Scheduler dispatches replacement.
- **NOZZLE_STUCK_OPEN**: Drone's nozzle jams during drop. Returns to base and goes permanently offline. Scheduler dispatches replacement.
- **ARRIVAL_SENSOR_FAILED**: Same transient recovery as DRONE_STUCK.

4.3.2 GUI-Based Fault Injection

1. In the **Fault Injection** panel (bottom-right of GUI), select a drone and fault type.
2. Click **Inject Fault**.
3. Observe the drone's state change in the table, map, and event log.

4.4 Unit Tests

- `DroneStateMachineJUnitTest.java` — State transition tests
- `FireIncidentSubsystemJUnitTest.java` — CSV parsing tests
- `UdpCommunicationJUnitTest.java` — Message serialization and UDP tests
- `PacketLossDetectionJUnitTest.java` — CRC32, sequence gap, resend tests
- `FaultDetectionJUnitTest.java` — Fault injection and recovery tests

4.5 Test Input Files

Event file (`Sample_event_file.csv`):

```
Time,Zone ID,Event type,Severity,Fault
14:03:15,3,FIRE_DETECTED,HIGH,DRONE_STUCK
14:10:00,7,DRONE_REQUEST,MODERATE,NONE
15:20:30,1,FIRE_DETECTED,LOW,NOZZLE_STUCK_OPEN
15:45:00,5,FIRE_DETECTED,HIGH,ARRIVAL_SENSOR_FAILED
16:00:15,2,DRONE_REQUEST,LOW,NONE
```

Zone file (`sample_zone_file.csv`):

```
Zone ID,Zone Start,Zone End
1,(0;0),(600;500)
2,(700;0),(1300;500)
3,(0;600),(600;1100)
4,(700;600),(1300;1100)
5,(0;1200),(600;1700)
6,(700;1200),(1300;1700)
7,(300;1800),(900;2300)
```

5 Results from Measurements

The Scheduler generates a metrics report at the end of each simulation, saved to `simulation_metrics_*.txt`

5.1 Required Performance Metrics

Table 1: Event response and completion times

Metric	Value
Average Event Response Time	47ms
Maximum Event Response Time	219ms
Average Event Completion Time	33s 766ms
Maximum Event Completion Time	56s 288ms

Table 2: Drone utilization (total simulation time: 5m 48s 733ms)

Drone	**Utilization**	**Drone**	**Utilization**
Drone-0	29.5%	Drone-10	14.2%
Drone-1	19.2%	Drone-11	29.1%
Drone-2	29.6%	Drone-12	33.2%
Drone-3	25.0%	Drone-13	31.8%
Drone-4	28.5%	Drone-14	11.3%
Drone-5	0.0%	Drone-15	14.6%
Drone-6	25.4%	Drone-16	9.5%
Drone-7	24.5%	Drone-17	15.7%
Drone-8	14.2%	Drone-18	40.3%
Drone-9	50.7%	Drone-19	11.3%

5.2 Simulation Overview

Table 3: Simulation overview (20 drones, 50 events)

Metric	**Value**
Incidents detected	50
Incidents extinguished	50
First fire simulated timestamp	00:03:32
Last fire extinguished simulated timestamp	07:54:45
Total simulation time	5m 48s 733ms
First incident to last extinguished	5m 34s 285ms
Total dispatch distance	87,026.02 m

Table 4: Per-drone metrics

Drone	Idle Time	Flight Time	Dispatch Dist. (m)	Fires	Water (L)
Drone-0	3m 30s 762ms	1m 42s 731ms	4600.69	3	70.00
Drone-1	4m 16s 677ms	1m 06s 820ms	5630.83	4	50.00
Drone-2	3m 40s 179ms	1m 43s 330ms	4777.16	2	50.00
Drone-3	3m 46s 260ms	1m 27s 241ms	6798.87	5	70.00
Drone-4	3m 39s 243ms	1m 39s 257ms	4580.39	2	60.00
Drone-5	5m 48s 525ms	0m 00s 000ms	0.00	0	0.00
Drone-6	3m 50s 081ms	1m 28s 424ms	4048.75	2	60.00
Drone-7	3m 58s 111ms	1m 25s 399ms	6338.94	3	50.00
Drone-8	4m 43s 924ms	0m 49s 583ms	2290.20	1	30.00
Drone-9	2m 01s 618ms	2m 56s 870ms	8097.29	4	100.00
Drone-10	4m 43s 904ms	0m 49s 617ms	2290.20	1	30.00
Drone-11	3m 27s 064ms	1m 41s 427ms	6246.72	4	80.00
Drone-12	3m 12s 658ms	1m 55s 828ms	5244.85	3	80.00
Drone-13	3m 27s 690ms	1m 50s 812ms	8294.26	4	60.00
Drone-14	4m 53s 937ms	0m 39s 575ms	2113.72	2	30.00
Drone-15	4m 37s 554ms	0m 50s 957ms	3235.88	2	40.00
Drone-16	5m 00s 441ms	0m 33s 075ms	1477.33	1	30.00
Drone-17	4m 28s 740ms	0m 54s 766ms	2394.95	2	50.00
Drone-18	2m 48s 089ms	2m 20s 408ms	6451.26	3	80.00
Drone-19	4m 53s 939ms	0m 39s 576ms	2113.72	2	30.00

6 Design Reflection

6.1 What We Like

- **CRC checksum:** Using a CRC checksum as a means to detect packet corruption makes it easy to verify if a packet was transmitted with correct information.
- **Centralized Message.java class:** Using a shared Message class for passing UDP messages between subsystems makes the networking much easier. The message class does the majority of the work, handling the message type, payload, packet loss detection, and checksum.
- **State machine:** The use of a drone state machine makes things much easier to implement with strict rules about what drones can and cannot do. The current state of the drone can be checked for when we want to perform certain actions, which makes implementing features simpler.
- **Fault injection:** Having features to inject fault (both via timed events in the csv file and manually from the frontend) makes it trivial to test the firefighting program's fault tolerance. We can simulate corrupting datagrams, dropping packets, drones getting

stuck, and the nozzle getting jammed, which would translate to the real-world scenario of this happening in a high-stakes situation.

- **Automatic metrics:** The Scheduler collects and reports performance metrics including per-drone and per-incident stats.

6.2 What Should Be Redone/Improved

- **Drone positioning:** Since real-world drones would report their position (not their progress on traveling in a straight path to a destination), it might make more sense to have drones report their X and Y positions in status updates, as opposed to the scheduler GUI making assumptions on position based on the drone's expected time to arrive at a zone

- **DroneSubsystem and Drones communicate via UDP:** Since drones would probably not have direct connection on the same device to the Drone Subsystem, it might make sense that drones are seperate programs entirely, which communicate via UDP packets to the Drone Subsystem.

- **CSV File Uploads:** Replacing the CSV files in the program makes takes extra steps to simulate different zones/fires, so having the simulation prompt the user with a file upload might be better

- **Zone-to-zone travel:** If a drone still has enough water to put out another fire, it might make sense to go directly to that zone if the battery allows. Implementing this to prevent unnecessary trips back and forth to the base station would decrease the time to put out fires (i.e. If 10L are left after putting out a fire, and there is a LOW intensity fire nearby)

- **Reimplement drone-stuck fault detection:** Currently the implementation used for detecting drone-stuck faults does not always work correctly, and sometimes unfaulty drones are detected as faulty due to arriving slightly late. Reimplementing the fault detection for this and using a standardized expected TTA for drones, with a small timeout might prevent this.